סיכום שיעור 7: Helm ומשאבים מתקדמים

Nov 13, 2026 · @Rivka

התמונה הגדולה

בשיעור 7 לקחנו את 12 הקבצים משיעור 6, וארזנו אותם לחבילה אחת עם Helm. בסוף השיעור רצו שתי ShopList זו לצד זו: אחת משיעור 6 בדלת 30080, ואחת מהחבילה בדלת 30081. בחלק השני הכרנו עוד ארבעה משאבים של Kubernetes.

הבעיה: 12 קבצים, וכל ערך כתוב ביד

המנהלת מבקשת: "תקימי עוד עותק של ShopList, לבדיקות". מה צריך לשנות?

- את ה-namespace, שכתוב ב-11 קבצים.
- את הדלת, nodePort, כי שתי אפליקציות לא יכולות לשבת על אותה דלת.
- את מספר העותקים, בשלושה מקומות.
- את הסיסמה, שכתובה גם ב-Secret וגם בבדיקה של postgres.

בערך 17 מקומות, ביד. ויש עוד שתי בעיות: סביבות שונות צריכות תיקיות כמעט זהות שמתרחקות עם הזמן, ואין כפתור "בטלי".

מנהל החבילות של Kubernetes

כמו ש-pip מתקין ספרייה של Python, הכלי Helm מתקין אפליקציה שלמה ב-Kubernetes: כל ה-Deployments, כל ה-Services, ה-Secret וה-PVC, ביחד, כיחידה אחת עם שם. הוא רץ במחשב שלכן, כמו kubectl, ומדבר עם אותו cluster.

המשפט של השיעור: "התבנית קבועה. מה שמשתנה, גר ב-values."

בכל שורה שואלות: זה משתנה בין התקנה להתקנה? אם כן, זה חור, והערך עובר לקובץ values.yaml. אם לא, הוא נשאר כתוב בקובץ.

האנלוגיה: בית דפוס להזמנות חתונה

ב-Helm	בבית הדפוס
התיקייה templates	העיצוב עם הקווים הריקים
הסוגריים המסולסלים	הקו הריק
הקובץ values.yaml	טופס הפרטים

ב-Helm	בבית הדפוס
הפקודה helm template	"תראי לי לפני שמדפיסים"
ה-release	ההזמנות של זוג מסוים
הפקודה upgrade, ומספר חדש: revision	הדפסה מתוקנת
הפקודה rollback	חזרה להדפסה הקודמת

שלוש מילים, וזה כל אוצר המילים

- **המילה chart:** התבנית. תיקייה עם קבצים, שאפשר להתקין הרבה פעמים.
- **המילה values:** הערכים שממלאים בחורים. ל-chart יש ברירות מחדל, וכל התקנה יכולה לדרוס חלק מהן.
- **המילה release:** התקנה אחת של chart, עם שם, בתוך namespace. כל שינוי בה מקבל מספר: revision.

מ-chart אחד אפשר להתקין כמה releases: עיצוב אחד, כמה זוגות. אבל כל release הוא התקנה של chart אחד בלבד.

בונים את ה-chart

המבנה: תיקייה עם שלושה דברים

```
helm/
  shoplist/
    Chart.yaml       # ID card: name and version
    values.yaml      # the form: default values
    templates/       # the 11 files from lesson 6, with holes
```

בתיקייה templates יש 11 קבצים, לא 12. הקובץ namespace.yml יצא, כי את ה-namespace בוחרים בזמן ההתקנה.

שתי גרסאות בקובץ Chart.yaml

- **השדה version:** הגרסה של האריזה, ה-chart עצמו. שיניתן קובץ ב-templates או ב-values.yaml? מעלים אותו.
- **השדה appVersion:** הגרסה של האפליקציה שבפנים, של ShopList. הוא רק מידע, ולא קובע איזה image יותקן.

השדה appVersion	השדה version	מה שינינו
לא משתנה	עולה, מ-0.1.0 ל-0.2.0	הוספנו לחבילה גיבוי לילי
עולה, מ-1.0.0 ל-1.1.0	אפשר לעדכן גם אותו	המפתחות הוסיפו חיפוש מוצרים ל-ShopList

איך קוראים חור

```
replicas: {{ .Values.backend.replicas }}
```

- הסוגריים המסולסלים: כאן יש חור, Helm ממלא אותו.
- המילה Values, עם V גדולה: לחפש בטופס, values.yaml.
- המילה backend: המגירה. המילה replicas: מה שבתוך המגירה.

כל נקודה היא קומה אחת למטה, בדיוק כמו ההזחה ב-values.yaml:

```
# values.yaml
imagePullPolicy: Never
backend:
  replicas: 1
  image:
    repository: shoplist-backend
    tag: local
```

המלכודת השקטה: אם כותבים replica בלי s, Helm לא נותן שגיאה. החור פשוט נשאר ריק, ו-Kubernetes משתמש ב-1. לכן תמיד מסתכלות על התוצאה של helm template.

הכתובת על המעטפה: Release

את ה-namespace לא שמים ב-values. הוא מגיע מפקודת ההתקנה, דרך הערכים ש-Helm ממלא לבד:

- הערך Release.Namespace: מה שכתבנו אחרי מינוס n.
- הערך Release.Name: השם שנתנו להתקנה.

ההזמנה לא כותבת על עצמה לאיזו עיר לשלוח. את הכתובת כותבים על המעטפה, ברגע השליחה.

הצינור: פס ייצור

```
POSTGRES_PASSWORD: {{ .Values.postgres.password | b64enc | quote }}
```

הערך נכנס משמאל ועובר תחנות: התחנה b64enc מקודדת ל-base64, והתחנה quote עוטפת במירכאות. בשיעור 6 קידדנו ביד, עכשיו Helm עושה את זה. ושם המשתמש שהופיע בשני מקומות קורא עכשיו מערך אחד.

ושוב: base64 זה לא הצפנה. סיסמה אמיתית לא נכנסת לגיט. בעבודה מעבירים אותה בזמן ההתקנה.

פתק "לא לפנות" על הארונית

```
metadata:
  name: postgres-pvc
  annotations:
    helm.sh/resource-policy: keep
```

בלי הפתק, הסרה עם Helm מוחקת גם את הארונית, ואת כל המוצרים. עם הפתק, הארונית נשארת. המדבקות מסוג labels הן לחיפוש, והפתקים מסוג annotations הם מידע לכלים כמו Helm.

שתי בדיקות לפני כל התקנה

- **הפקודה helm lint, המגיהה:** המבנה תקין, ה-YAML חוקי, ואפשר למלא את כל החורים.
- **הפקודה helm template, התצוגה לפני הדפסה:** מה בדיוק יגיע ל-cluster. היא לא נוגעת ב-cluster, אז אפשר להריץ אותה כמה שרוצים.

```
helm lint ./helm/shoplist
helm template shop ./helm/shoplist -n shop-helm
helm template shop ./helm/shoplist -s templates/backend-deployment.yml --set
backend.replicas=3
```

החיים של release

מה קורה בתוך התקנה

1. ממזג ערכים: קודם values.yaml, אחר כך קבצים עם מינוס f, ואחר כך set.
2. ממלא את החורים, בדיוק כמו helm template.
3. שולח ל-cluster, לאותו מקום ש-kubectl apply שולח.
4. רושם בקלסר: כל revision נשמר כ-Secret באותו namespace. אין שרת של Helm.

על כל משאב שהוא יוצר, Helm מדביק מדבקת בעלות. מעכשיו הוא נוגע רק במה שיש עליו המדבקה שלו.

התקנה

```
helm install shop ./helm/shoplist -n shop-helm --create-namespace --set
frontend.nodePort=30081
```

מה הוא אומר	החלק בפקודה
התקנה חדשה, ושם ה-release הוא shop	המילים install shop
מאיזה chart להתקין	הנתיב ./helm/shoplist.
לאיזה namespace. זה מה שנכנס ל-Release.Namespace	הדגל מינוס n
ליצור את ה-namespace אם הוא לא קיים	הדגל create-namespace
כי 30080 תפוסה על ידי שיעור 6	הדגל set עם הדלת 30081

מוצר שמוסיפים ב-30081 לא מופיע ב-30080: שתי ארוניות, שני מסדי נתונים, מאותו chart.

שתי התקנות שנכשלות: בלי הדלת 30081, כי הדלת תפוסה. ולתוך ה-namespace של שיעור 6, כי Helm לא דורס משאב שהוא לא יצר.

מה Helm יודע

```
helm list -A                          # all releases (not charts!)
helm status shop -n shop-helm         # status and current revision
helm get values shop -n shop-helm     # only what we asked for
kubectl get secrets -n shop-helm      # sh.helm.release.v1.shop.v1 = the
binder
```

עדכון: מה שלא כתבת, חוזר לברירת המחדל

בעדכון, כל ערך שלא עובר שוב בפקודה חוזר לערך שב-values.yaml. עדכון עם set של מספר העותקים בלבד מחזיר את הדלת ל-30080, ונכשל. כל הזמנה היא הזמנה מלאה.

הפתרון: טופס משלכן, בקובץ, עם רק מה ששונה מברירת המחדל:

```
# helm/shoplist/values-test.yaml
frontend:
  nodePort: 30081
backend:
  replicas: 3
```

```
helm upgrade shop ./helm/shoplist -n shop-helm -f helm/shoplist/values-
test.yaml
```

קובץ נשמר בגיט, ורואים בדיוק מה ביקשנו. את ה-set שוכחים.

מי מנצח? מי שמדבר אחרון

1. ברירת המחדל של בית הדפוס: הקובץ values.yaml שבתוך ה-chart.
2. הטופס שהזוג מילא: קובץ עם מינוס f. אם יש כמה, האחרון מנצח.
3. הטלפון של הרגע האחרון: set. הוא דורס את כולם.

ככה פותרים סביבות: chart אחד, ושני טפסים קטנים, למשל values-dev.yaml ו-values-prod.yaml.

לשבור, ולחזור אחורה

```
helm upgrade shop ./helm/shoplist -n shop-helm -f helm/shoplist/values-
test.yaml --set backend.image.tag=oops
helm history shop -n shop-helm
helm rollback shop 2 -n shop-helm
helm history shop -n shop-helm      # revision 4: Rollback to 2
```

האפליקציה לא נופלת: ה-pod החדש אף פעם לא מוכן, אז הישנים לא יורדים. זו בדיקת המוכנות משיעור 6.

למה 4 ולא 2? כמו היסטוריית גרסאות ב-Google Docs: שחזור של גרסה ישנה יוצר גרסה חדשה. ההיסטוריה לא נמחקת.

הסרה: החדר מתרוקן, הארונית נשארת

```
helm uninstall shop -n shop-helm
kubectl get all,pvc -n shop-helm     # only postgres-pvc is left
helm install shop ./helm/shoplist -n shop-helm -f helm/shoplist/values-
test.yaml
```

אחרי התקנה מחדש המוצרים חוזרים, כי הארונית נשארה. מחיקה של ה-namespace עם kubectl, לעומת זאת, מוחקת הכל, כולל הארונית.

הכלל: release ש-Helm התקין, משנים רק דרך Helm. שינוי עם kubectl לא נרשם בקלסר, והעדכון הבא ידרוס אותו.

משאבים מתקדמים

חוזרות למטבח של הקונדיטוריה משיעור 5: כל node הוא מטבח, כל pod הוא מגש, וה-scheduler הוא הסדרנית.

במטבח	מה הוא פותר	המשאב
מקום שמור במטבח, ותקרה	כמה מעבד וזיכרון כל container מקבל	השדות requests ו-limits
הזמנה חד-פעמית: עוגת יום הולדת	משימה שרצה פעם אחת ונגמרת	המשאב Job
משימה ביומן המטבח: ספירת מלאי כל לילה	משימה שרצה לפי שעון	המשאב CronJob
מגשים ממוספרים, לכל אחד ארונית	עם שם קבוע ודיסק משלהם	המשאב StatefulSet
לפני חג: עוד מגשים	מוסיף ומוריד pods לפי עומס	המשאב HPA

מקום שמור במטבח, ותקרה

- **השדה requests, המקום השמור:** הסדרנית שולחת מגש רק למטבח שנשאר בו מספיק מקום.
- **השדה limits, התקרה:** זמן תנור (מעבד) מעל התקרה? מחכה בתור ומאט. מקום על השיש (זיכרון) מעל התקרה? המגש נזרק, OOMKilled, ומכינים חדש.

```
resources:
  requests:
    cpu: 100m        # a tenth of a core
    memory: 128Mi
  limits:
    cpu: 500m        # half a core
    memory: 256Mi
```

בהדגמה נתנו ל-backend תקרת זיכרון של 20Mi. ה-pod החדש נזרק שוב ושוב, והאפליקציה המשיכה לעבוד בזכות המגשים הישנים. חזרה אחורה תיקנה הכל.

הזמנה חד-פעמית, ומשימה ביומן

- ה-Deployment הוא ההזמנה הקבועה: נגמר מגש? מכינים חדש. לכן גיבוי בתוכו היה רץ לנצח.
- ה-Job הוא הזמנה חד-פעמית: אופים, מגישים, ונסגר. נכשל? מנסה שוב, עד backoffLimit.
- ה-CronJob הוא משימה ביומן: בכל פעם שהשעה מגיעה, נפתח Job חדש.

הזמנים כתובים בחמישה שדות: דקה, שעה, יום בחודש, חודש, יום בשבוע. הביטוי `0 2 * * *` פירושו כל לילה בשתיים, לפי השעון של ה-cluster, שהוא לרוב UTC.

ב-ShopList: גיבוי לילי של מסד הנתונים, עם pg_dump, לארונית נפרדת בשם backup-pvc. שלושה דברים חדשים בקובץ:

1. חור מסוג חדש, תנאי: `{{- if .Values.backup.enabled }}`. אם false, הקובץ לא קיים בכלל.
2. שלוש קומות: ה-CronJob יוצר Job, וה-Job יוצר pod.
3. מדיניות הפעלה מחדש OnFailure: לנסות שוב רק אם נכשל.

ושלושה שמות מקבצים אחרים, שחייבים להתאים: postgres-secret, ה-Service בשם postgres, ו-backup-pvc.

כדי להפעיל את הגיבוי עכשיו, בלי לחכות ללילה:

```
kubectl create job backup-now --from=cronjob/backup -n shop-helm
kubectl logs job/backup-now -n shop-helm     # backup saved: ...
```

כאן המצב Completed הוא בדיוק המטרה. וגיבוי שלא בדקו שאפשר לשחזר ממנו, הוא לא גיבוי.

מגשים זהים, ומגשים ממוספרים (רק להכיר)

ה-StatefulSet: מגשים ממוספרים	ה-Deployment: מגשים זהים	
קבועים: postgres-0, postgres-1	אקראיים	שמות
ארונית לכל pod	אחת משותפת, או בלי	ארונית
אחד אחרי השני	כולם בבת אחת	עלייה
חוזר עם אותו שם ואותה ארונית	חוזר עם שם חדש	שנמחק pod
מסדי נתונים עם כמה עותקים	ה-backend, ה-frontend, ה-stats	מתאים ל

ב-ShopList יש postgres אחד עם ארונית אחת, אז Deployment מספיק. בעבודה משתמשים בדרך כלל ב-chart מוכן, או במסד נתונים מנוהל בענן.

לפני חג: עוד מגשים (הגדלה אוטומטית)

ה-HPA מודד כל כמה שניות, ומשנה את מספר העותקים של ה-Deployment, בין מינימום למקסימום.

- היעד, למשל 70 אחוז, מחושב מתוך ה-requests. בלי requests אין הגדלה אוטומטית.
- מי שמודד הוא רכיב בשם metrics-server. בענן הוא כמעט תמיד מותקן. ב-Docker Desktop לא, ולכן ב-chart שלנו ההגדלה כבויה כברירת מחדל.

מילון פקודות: kubectl מול Helm

עם Helm	עם kubectl, בשיעור 6	מה רוצות
`helm template shop ./helm/shoplist`	אין	לראות לפני שמתקינים
`helm lint ./helm/shoplist`	אין	לבדוק שהכל תקין
`helm install shop ./helm/shoplist -n <ns> --create-namespace`	`kubectl apply -f kubernetes/`	להתקין
`helm upgrade shop ./helm/shoplist -n <ns> -f <file>`	לערוך קובץ, ולהחיל שוב	לשנות ערך
`helm list -A`	`kubectl get all -n <ns>`	לראות מה מותקן
`helm history shop -n <ns>`	אין	היסטוריה
`helm rollback shop 2 -n <ns>`	אין, רק ביד	לחזור אחורה
`helm uninstall shop -n <ns>`	`kubectl delete -f kubernetes/`	להסיר

ועוד פקודות מהחלק השני:

```
kubectl describe deployment backend -n shop-helm | grep -A6 Limits   #
requests and limits
kubectl get cronjob -n shop-helm                                      # the
nightly backup
kubectl create job backup-now --from=cronjob/backup -n shop-helm      # run
it now
kubectl get hpa -n shop-helm                                          #
autoscaling, if enabled
```

תקלות נפוצות

מה עושים	הסיבה	מה רואים
לסגור ולפתוח טרמינל, ולבדוק עם `helm version`	לא מותקן, או שהטרמינל Helm נפתח לפני ההתקנה	ההודעה ש-helm לא מוכר

מה עושים	הסיבה	מה רואים
בשגיאה כתובים הקובץ והשורה	טעות כתיב באמצע הכתובת, כמו imag	השגיאה nil pointer
אין שגיאה! להשוות לשם שב-values.yaml	טעות כתיב בסוף הכתובת, כמו replica	שורה ריקה ב-helm template
דלת אחרת, בקובץ values	ה-nodePort תפוס בהתקנה אחרת	הדלת תפוסה, port already allocated
להשתמש ב-upgrade במקום install	כבר יש release בשם הזה	השם כבר בשימוש
להתקין ל-namespace אחר	המשאב נוצר עם kubectl, בלי מדבקת הבעלות של Helm	השגיאה cannot be imported
ערכים קבועים שמים בקובץ עם מינוס f	ערך שעבר ב-set בפעם הקודמת, ולא הפעם	העדכון החזיר ערך ישן
להגדיל את ה-limit ב-values	ה-container עבר את תקרת הזיכרון	המצב OOMKilled
להקטין את ה-requests	ה-requests גדולים ממה שיש במטבח	ה-pod תקוע ב-Pending עם Insufficient
לקרוא את הלוג של ה-Job	אחד משלושת השמות לא תואם	ה-pod של הגיבוי ב-Error

בעולם האמיתי

- **חנות ה-charts:** לרוב התוכנות המוכרות יש chart מוכן, באתר Artifact Hub. תמיד קודם קוראים את הטופס שלו עם `helm show values`, ורק אז מתקינים.
- **הפייפליין:** בשיעור 8 ההתקנה תהיה שורה אחת, עדכון עם הדגל install: מתקין בפעם הראשונה, מעדכן בכל פעם אחרת. הפייפליין משנה רק את התגית של ה-image.
- **מתי מספיק YAML:** כשלומדות, או משהו קטן שמותקן פעם אחת. כדאי לעבור ל-Helm כשיש כמה סביבות, כשצריך היסטוריה וחזרה אחורה, וכשמתקינים תוכנה מוכרת.
- **רק לדעת שקיים:** הכלי Kustomize פותר את אותה בעיה בלי חורים, בלי היסטוריה ובלי חזרה אחורה.

שאלות לחזרה ולראיון

קודם לנסות לענות בעל פה, ורק אז לקרוא.

1. **מה זה Helm, ואיזו בעיה הוא פותר?** מנהל חבילות ל-Kubernetes. הוא אורז את כל הקבצים ל-chart עם ערכים, ופותר שכפול בין סביבות, התקנה כיחידה אחת, והיסטוריה עם חזרה אחורה.
2. **מה ההבדל בין chart, release ו-revision?** התבנית, התקנה אחת עם שם, וגרסה של התקנה.
3. **מי מנצח כשאותו ערך מופיע בכמה מקומות?** קודם values.yaml, אחר כך קבצי מינוס f לפי הסדר, ואחר כך set. ובעדכון, מה שלא עובר שוב חוזר לברירת המחדל.
4. **מה קורה בחזרה אחורה, ולמה נוצר revision חדש?** Helm מחיל שוב את התוכן הישן ורושם אותו כגרסה חדשה, כדי שההיסטוריה תישאר מלאה.
5. **מה קורה ל-PVC בהסרה?** נמחק עם הנתונים, אלא אם יש עליו את הפתק keep.
6. **איפה Helm שומר את ההיסטוריה?** בתוך ה-cluster, כ-Secrets באותו namespace. אין שרת נפרד.
7. **מה ההבדל בין requests ל-limits?** הראשון מקום שמור שלפיו הסדרנית משבצת, והשני תקרה. מעל תקרת המעבד מאטים, ומעל תקרת הזיכרון ה-container נהרג.
8. **מה ההבדל בין Deployment, Job ו-CronJob?** רץ תמיד, רץ פעם אחת עד שמצליח, ורץ לפי לוח זמנים.
9. **מתי StatefulSet ולא Deployment?** כשכל pod צריך שם קבוע וארונית משלו, כמו במסד נתונים עם כמה עותקים.
10. **איך ההגדלה האוטומטית מחליטה כמה pods צריך?** היא משווה את העומס ליעד, באחוזים מה-requests, ומשנה את מספר העותקים בין מינימום למקסימום.

מה הלאה

- **תרגולים לבית:** הקובץ exercises.md בתיקייה של שיעור 7. שישה תרגולים עם פתרון מקופל. לא מוגש.
- **שיעורי בית:** הקובץ homework.md. לבנות את ה-chart בבית, להתקין, לעדכן, לשבור, לחזור אחורה, ולהוסיף מקום שמור, תקרה וגיבוי לילי. הגשה בדחיפה מהבית, לפני מפגש 8.
- **בשיעור הבא:** CI/CD. הפייפליין יבנה את ה-images ויריץ את העדכון עם Helm לבד, בכל דחיפה.